DAY 09 · BEGINNER · DOCKER FROM TODAY

Documents, Chunks and Vectors

Cut a document up, turn it into numbers, and find the right paragraph by meaning. Then end on a failure.

RecursiveCharacterTextSplitter | FastEmbed | Qdrant | cosine | Notebook

KGP Talkie
LEARN. BUILD. TALK AI

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

The 10-Day Course Map

WEEK 1 TALKING TO A MODEL

01 Your First AI Program
Ask a model, print the answer

02 Roles and Messages
The system message sets the job

03 Prompt Templates
One template, many inputs

YOU WILL HAVE BUILT
a script that answers anything you type into it

WEEK 2 BUILDING SOMETHING

04 Chains: The Pipe
`prompt | model | parser`

05 Structured Output
An object, not a paragraph

06 Memory
A checkpointer and a thread id

YOU WILL HAVE BUILT
a chatbot that remembers what you said a minute ago

WEEK 3 GIVING IT ABILITIES

07 Tools
Your Python, called by the model

08 Your First Agent
It decides when to call them

YOU ARE HERE
09 Chunks and Vectors
Find a paragraph by meaning

10 RAG with RAGWire
Citations, and a refusal

One notebook and one deck a day, **60 to 90 minutes** each. Every model used is a **free** one, and Docker only turns up on Day 9.

Why Not Send the Whole Document

A 300-page manual



the answer is one paragraph of it

Send all of it?

It will not fit in the context window

And you pay for all of it, every turn

Send the part that matters

"Economy fare includes 15 kg of checked baggage."

The whole problem is **finding that part.**

Today is about finding that part **without keyword search**.

Choosing a Chunk Size

chunk_size=100

"## Checked baggage"

"Economy fare includes 15 kg"

The heading is separated from the rule underneath.

Retrieve one and you have neither.

chunk_size=250

"## Checked baggage
Economy fare includes 15 kg, Business 30 kg."

One idea, with its heading and its context intact.

250 to 800 is a reasonable start

chunk_size=1000

cabin baggage + checked baggage + excess fees + delayed baggage + claims

Retrieval returns everything.

The matching part gets diluted.

`chunk_overlap` repeats a little of the previous piece, so a sentence straddling a boundary is not lost to both sides.

Embeddings: Text as Numbers

"How much baggage can I check in?"

embed →

```
[-0.0316, 0.0442, -0.0119, 0.0287, -0.0538,
0.0091, 0.0374, -0.0205, ... ]
```

384 numbers, and you will never read them

What matters is that **two sentences with similar meaning get similar numbers**,

even when they share no words at all.

This model runs on your own CPU and needs **no API key**.

THE MATCH SHARES NO WORDS WITH THE QUESTION

Search by Meaning, Not Keywords

05

KGP Talkie
LEARN. BUILD. TALK AI

THE QUESTION

"How much **luggage** is included with an **economy ticket**?"

THE CHUNK IT FOUND

"**Economy fare** includes 15 kg of **checked baggage**."

luggage / baggage. ticket / fare. **A keyword search would have returned nothing at all.**

RANKED BY SIMILARITY

0.811	## Checked baggage Economy fare includes 15 kg...
0.738	# Zephyr Airlines Baggage Policy 2026 ## Cabin baggage...

The scores sit close together. The **order** is what you use, not the value.

Retrieval Always Returns Something

"What is the refund policy for cancelled flights?"

asked against a document with no refund section

0.614	## Delayed baggage Report delayed baggage at the arrival desk...
0.566	## Checked baggage Economy fare includes 15 kg...

No error. No warning. No empty result. **There is no concept of "not in here".**

Hand that chunk to a model and it writes about refunds, from a baggage policy.

Ask for the nearest 3 and you get 3, **whatever you asked**. Tomorrow starts here.